0.1 Collaboration Statement

The last page of the course syllabus contains a statement of the department collaboration policy with regard to projects. Violation of the collaboration policy can result in failure of the course.

0.2 Reminder: Project Hand in Rules

Read and follow the instructions!

1. All project due dates are specified on Brightspace.
2. All projects handed in must be NetBeans projects.
3. Each student starts the semester with 7 free late days. When handing in a project late you must add a comment telling me that you are using some number of late days for the project.
4. If you hand in a project late, you are not allowed to correct and resubmit. This is the only penalty for late submissions.
5. For projects, you will hand in what is specified by individual assignments. Most of the time it will be an entire zipped project folder. Name the archive with ONLY YOUR LAST NAME. Example: Smith.zip ONLY zip archives will be accepted.

0.3 Objectives

1. Create a class to hold the data.
2. Create an array to hold the objects.
3. Do some searching and summing and summarize the data.
4. Output all requested information with meaningful labels in an understandable format.
5. Use formatted numerical output.

0.4 Grading Criteria:

Grading for this project will be done as follows:

Grade Earned	Requirements
D	The main method is the only static method in the class. The main method checks the command line arguments. Usage message is correct for the number of required arguments. The main method in the driver creates an object of the class and calls a run() method passing the arguments. All code handed in is syntactically correct and correctly formatted. The program compiles and runs.
C	Everything above plus Create class to hold the data. Declare array to hold the objects. Read and store the data. Write methods for statistics. Display the information in a pleasing form. All methods have complete JavaDoc.
B	Everything above plus: Find and store starting indexes. Find and store first fall frost dates. Correctly display each transaction as a series of formatted Strings. All methods have complete JavaDoc
A	Everything above plus: Find and display the average first fall frost day of the year by decade. Display an analysis of the averages. Revision History is complete All methods have complete JavaDoc

NOTE: The requirements for a D project are exactly the same on all projects.

Zip the entire project and upload it on Brightspace.

1 Introduction

In this project you will write a class to hold the data for a single record of climate data. In this assignment you are going to write a program to process weather data from the National Climate Data Center. The NCDC maintains a data collection of daily weather observations for more than 19,000 locations in the United States. Some of these have been recorded since the late 19th century. The data used in this assignment was obtained from their web site www.ncdc.noaa.gov/cdo-web.

Create a Project called ClimateData and download the data file into your project directory. Also download the file Date.java an put it into the `ClimateData/src/climatedata/` directory. This contains a condensed version of the daily weather data recorded at the Portland International Jetport since 1941. The data has been sanitized i.e. all incomplete data readings have been deleted so that you will have no problem

parsing the data. You are going to be analyzing this data to look for evidence of climate change.

2 Input file format

Data files come in many different formats and sometimes need different techniques to parse them. This file is very simple. It is organized by line and in fixed width columns. The first three lines are a header describing the size of the file and the data columns. Each subsequent line represents a single day's weather record. The columns are:

DATE - The date of the observation in format MM/DD/YYYY

TMAX - The maximum temperature in Fahrenheit

TMIN - The minimum temperature in Fahrenheit

The first few lines of the file are shown below:

```
29220 data records
DATE       TMAX TMIN
---------- ---- ----
01/01/1941   38   25
01/02/1941   32   20
01/03/1941   31   22
01/04/1941   34   25
01/05/1941   32   20
01/06/1941   29    5
01/07/1941   29  -10
```

3 Your Tasks

1. Create a class to hold a single reading. Call it DailyReading. Its private properties are a Date called date, an integer maxTemp, and an integer minTemp. It needs a parameterized constructor, accessors (getters) for each of the 3 properties, and a toString() which is simply this:

   ```java
   @Override
   public String toString() {
       return String.format("%s %6d %6d", date.toString(),tMax, tMin);
   }
   ```

2. The ClimateData.java (the driver for the program) will have several properties (your variable names need not match these):

```
private DailyReading[] readings;
private double averageMaxTemp;
private double averageMinTemp;
private int[] yearStartingIndexes;
```

Declare the arrays, do NOT create them.

3. The method that reads in the file will create the readings array after reading the number of records from the first line of the file. Read that number of records and then create the array for the DailyReading objects. This will hold all the data from the file. This is a lot of data, but your computer can easily hold it in memory.

4. Read and Store the Data in your Array. First call your Scanner's nextLine() method 3 times to discard the remainder of the 3 header lines. Next you are going to read and store all the data into your array. The Scanner's default behavior is to break up data at whitespace (spaces, tabs, and newlines), but we also want to break apart the date string which uses forward slashes to separate the month, day and year. Use code like this to change the delimiter to also include forward slashes:

```
fileScnr.useDelimiter("[/ \t\n\r]+");
```

   NOTE: There is a space between the forward slash and the first backslash. Now you can just use .nextInt() to read the five fields: month, day, year, tmax, and tmin. Create a new DailyReading using those inputs and place it into the next element of the array. The Scanner will then automatically wrap to the next line and you can continue reading the entire file, and storing it in your array. Since you know how many records there are you can use a for loop instead of a while loop.

   Verify that you have read in the data correctly. Either examine your array using the debugger or print them. Do they match the contents of the file?

5. Long Term Averages and Records

   (a) Write a method that finds and returns the index of the highest tMax value in the array.

   (b) Write a method that finds and returns the index of the lowest tMin value in the array.

   Test your output against the example output below.

   Once these are working correctly modify the method that finds the maximum and add the code necessary to find the average high temp while finding the maximum temperature and modify the method that finds the minimum to find the average low temp while finding the minimum temperature.

   Use those methods from run() to find and print:

   (a) The highest temperature and the date it occurred on

(b) The lowest temperature and the date it occurred on

(c) The average maximum temperature (Check your value. It should be 55.6608...)

(d) The average minimum temperature

6. Finding the starting index for each year. One aspect of climate change that has been noted in Maine is that the first fall frost has been occurring later in the year. We are going to analyze the minimum daily temperature data to calculate the date each fall when the temperature first reaches freezing temperatures. First we are going to find the starting index of each year in the readings array. Your task is to create a loop that prints:

year	starting index
1941	0
1942	365
1943	730
1944	1095
1945	1461

Remember it is poor programming to hard code specific values such as 1941 and 2020 in a program because then the program would have to be modified to process a different file. Instead use year[0] to get the starting year and year[n-1] to get the last year. (where the variable n is the number of data records.) Store these values in property `yearStartingIndexes`. You should be able to figure out how long to make the array. You know how to access the first year and the last value and there are 12 months in each year. Although there might be missing data, you can still make a good guess at the length of the needed array.

7. Finding the first fall frost. Write a method
`int firstFallFrost(int yearStartIndex, int yearEndIndex)`
that looks at a one year section of the array for the date of the first fall frost and returns the index of that day. Hint: start searching forward from the middle of the year. For testing, you should find that for 1941 (0, 365) it occurs at index 262, and for 1942 (365, 730) it occurs at index 636. In the next part you will print the human understandable dates 9/20 and 9/29.

8. Printing the first fall frost dates. Use your starting year indices from above and your firstFallFrost() method from above, to print a table of the first fall frost dates for each year like this:

year	first fall frost
1941	9/20
1942	9/29
1943	10/10
1944	9/25
1945	9/30

Make sure your results match.

9. Averaging by decade: A nicer way to present the data is averaged by decade, i.e. the average fall frost day in the 1940's, 1950's, ... 2010's. Write code to do the analyses

this way. Note that for some decades we may not have data for all 10 years (in our data the 1940's and the 2020's). Your code should still give correct averages for these decades. Print your results in a neat table. NOTE: The Date Class has a property called day number and a method that calculates the difference (in days) between two dates. The first day of the year where the frost date occurs can be calculated using the difference between January first of a year and the date where that first frost occurred.

10. Analysis of the results: The first fall frost dates vary a lot from year to year. Compare the average of the first 10 years of the historical record to the average of the most recent 10 years? How much has it changed? Have your program print a short statement about the observed trend. Again none of the numbers or the word "increased" or "decreased" should be hard coded. Use a decision statement with appropriate conditions to put that statement together.

Actual Program output if all parts are completed:

```
run:
The data covers the time period from 01/01/1941 to 12/31/2020.
The highest temperature in the data is: 103 degrees F which occurred on 08/02/1975.
The lowest temperature in the data is: -39 degrees F which occurred on 02/16/1943.
The average high temperature was 55.66 degrees F.
The average low temperature was 36.27 degrees F.

Number of years: 2020 - 1941 + 1 = 80
Year   Starting Index
1941              0
1942            365
1943            730
1944           1095
1945           1461
1946           1826
1947           2191
1948           2556
1949           2922
1950           3287
. . . output missing here
2016          27393
2017          27759
2018          28124
2019          28489
2020          28854

Year        First Fall Frost
1941            09/20
1942            09/29
1943            10/10
1944            09/25
1945            09/30
1946            10/03
1947            09/24
1948            09/17
1949            09/26
 . . . output missing here
2014            11/03
2015            10/17
2016            10/15
2017            11/01
2018            10/18
2019            10/05
2020            10/09

Decade         Average DoY    Approx
             of First Frost    Date
1940's           271          08/26
1950's           270          08/25
1960's           272          08/26
1970's           279          09/03
1980's           283          09/07
1990's           280          09/05
2000's           285          09/09
2010's           292          09/16
Over the 8 decades of data the first fall frost date has increased by 21 days.
BUILD SUCCESSFUL (total time: 0 seconds)
```

HELP! If there is something in the instructions that you find difficult to understand, or something that is misleading or contradictory, please let me know and I will try to rewrite

that instruction. You need to let me know exactly which paragraph the offending instruction is in. Thanks!